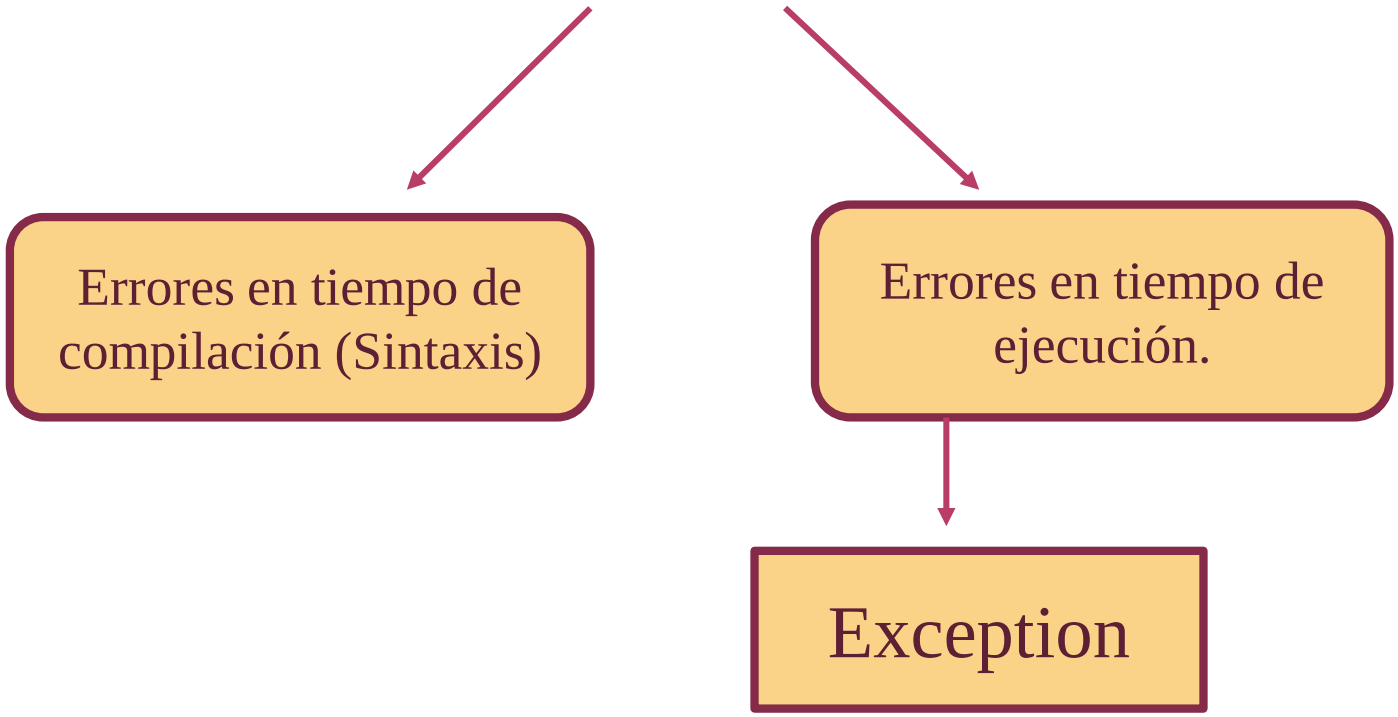
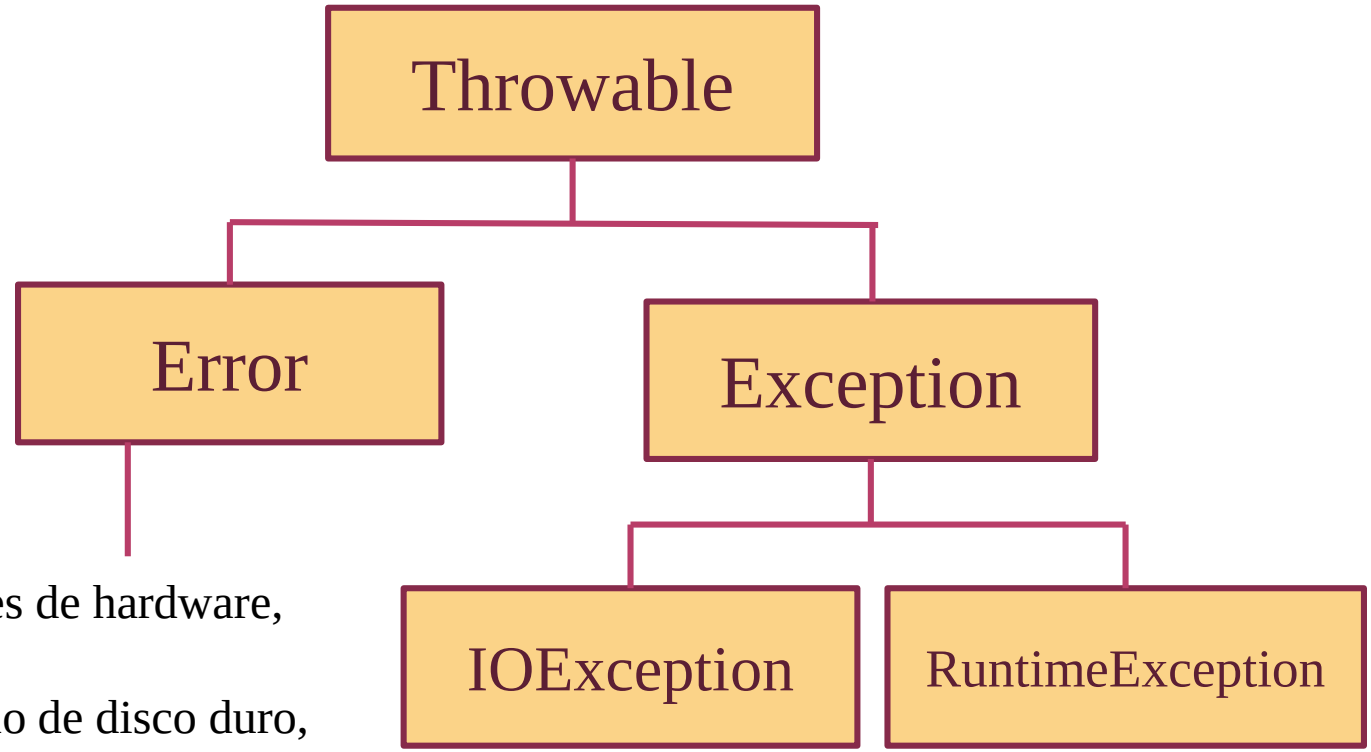
EXCEPCIONES ARCHIVOS

Autor: Tamara Herrera

TIPOS DE ERRORES



EXCEPCIONES



Errores de hardware, poco
espacio de disco duro, memoria, etc.

TIPOS DE DE EXCEPTION

IOException

Son las excepciones comprobadas. Son errores que no son culpa del programador. Por ej: no se encuentra una imagen en una ruta determinada. Java te obliga a trabajar con el bloque Try Catch

RuntimeException

Son las excepciones no comprobadas. Son excepciones que el programador puede mejorar, por ejemplo que una variable este nula. Java no te obliga a trabajar con Try y Catch.

IOEXCEPTION

Si todo va bien devuelve el BufferedImage sino devuelve el IOException

read

```
public static BufferedImage read(File input)
                              throws IOException
```

Returns a `BufferedImage` as the result of decoding a supplied `File` with an `ImageReader` chosen automatically from among those currently registered. The `File` is wrapped in an `ImageInputStream`. If no registered `ImageReader` claims to be able to read the resulting stream, `null` is returned.

The current cache settings from `getUseCache`and `getCacheDirectory` will be used to control caching in the `ImageInputStream` that is created.

Note that there is no `read` method that takes a filename as a `String`; use this method instead after creating a `File` from the filename.

This method does not attempt to locate `ImageReaders` that can read directly from a `File`; that may be accomplished using `IIORegistry` and `ImageReaderSpi`.

Parameters:

`input` - a `File` to read from.

Returns:

a `BufferedImage` containing the decoded contents of the input, or `null`.

Throws:

`IllegalArgumentException` - if `input` is `null`.

`IOException` - if an error occurs during reading.

TIPOS DE DE EXCEPTION

IOException

Son las excepciones comprobadas. Son errores que no son culpa del programador. Por ej: no se encuentra una imagen en una ruta determinada. Java te obliga a trabajar con el bloque Try Catch

RuntimeException

Son las excepciones no comprobadas. Son excepciones que el programador puede mejorar, por ejemplo que una variable este nula. Java no te obliga a trabajar con Try y Catch.

ARCHIVOS

Autor: Tamara Herrera

ARCHIVOS

Permite guarda información en forma de archivo en nuestra página

Flujo de Bytes

1010101010101
0101010100101
0101001010101

Utilizamos las clases

- InputStream
- OutputStream

Flujo de caracteres

AJSKJAJSJFSA
JDNJSNCJSNA
KCSNCNJKA1

Utilizamos las clases

- FileReader
- BufferedReader
- FileWriter
- BufferedWriter

COMPROBAR SI EXISTE UN ARCHIVO

Puede recibir ruta absoluta o ruta relativa

```
import java.io.File;
```

```
public boolean existe(String ruta)
{
    File archivo = new File(ruta);
    if(archivo.exists())
        return true;
    return false;
}
```

CREAR ARCHIVO

Parámetro que indica si va a anexar la info al archivo o sobreescribir

```
import java.io.FileWriter;

public boolean creaArchivo(String ruta)
{
    FileWriter escritura;
    try {
        escritura = new FileWriter(ruta, true);
        escritura.write("");
        escritura.close();
        return true;
    } catch (IOException e) {
        e.printStackTrace();
    }
    return false;
}
```

FILE READER Y FILEWRITER

Escribe y lee en el archivo, pero trabaja letra por letra

FILEWRITER

```
import java.io.FileWriter;

public void escribe_letra_x_letra(String frase) {
    try {
        FileWriter fw = new FileWriter(ruta, true);

        for (int i = 0; i < frase.length(); i++) {
                fw.write(frase.charAt(i));
            }
        //fw.write(33); //Puede escribir caracteres
        //fw.write(frase); //Puede escribir frases enteras
        fw.close();

    } catch (IOException e) {
        e.printStackTrace();
    }
}
```

FILEREADER

```
import java.io.FileReader;

 public void lee_letra_x_letra() {

     FileReader entrada;
     try {
         entrada = new FileReader(ruta);
         int c = entrada.read(); //Lee el primer caracter
         while (c != -1)
         {
             char letra = (char) c;
             System.out.println(letra);
             c = entrada.read();
         }
         entrada.close();

     } catch (Exception e) {
         e.printStackTrace();
     }
 }
```

BUFFERED READER Y BUFFERED WRITER

Escribe y lee en el archivo, pero trabaja por renglones

BUFFERS

Los Buffers permiten obtener información de a líneas y no de forma secuencial.

Eclipse

```
1  package Streams;
2
3  import java.io.FileNotFoundException;
6
7  public class Ejemplo1_LeerDatos {
8
9      public static void main(String[] args) {
10         leerFichero lf = new leerFichero();
11         lf.lee();
12     }
13
14 }
```

BUFFER

Test.txt

AJSKJAJSJFSA
JDNJSNCJSNA
KCSNCNJKA1

Acelera tanto la escritura como la lectura. Crea una memoria intermedia entre el archivo y eclipse.

BUFFERED WRITER

```
import java.io.BufferedWriter;

 public void escribe_lineas(String frase) {
     try
     {
         FileWriter entrada = new FileWriter(ruta, true);
         BufferedWriter miBuffer = new BufferedWriter(entrada);
         miBuffer.write(frase);
         miBuffer.close();
         entrada.close();

     } catch (IOException e) {
         e.printStackTrace();
     }
 }
```

BUFFERED READER

```
import java.io.BufferedReader;

 public void lee_lineas() {
     FileReader entrada;
     try {
         entrada = new FileReader(ruta);
         BufferedReader miBuffer = new BufferedReader(entrada);

        String linea = "";
         while (linea != null) {
             System.out.println(linea);
             linea = miBuffer.readLine();
         }

         entrada.close();

     } catch (IOException e) {
         System.out.println("No se encontro el archivo");
     }
 }
```